Differentiation on a Computer I

From wobbly secants to exact derivatives

Prof. Nipun Batra

Mathematical Foundations for AI · IIT Gandhinagar

The hook: how PyTorch checks its own homework

The bug that does not crash

You extend PyTorch with a custom operation — a fused kernel, a new layer. Autograd asks **you** to supply its derivative.

Get it slightly wrong, and **nothing crashes**:

- the loss still goes down (mostly) — wrong gradients are often “downhill-ish”
- the model trains, ships… and is quietly worse than it should be

> A wrong gradient is the nastiest bug class in ML: no exception, no NaN, no stack trace — just a model that underperforms for a reason you can’t see.

The referee: `torch.autograd.gradcheck`

Every serious framework ships a gradient checker. PyTorch's:

```
import torch
from torch.autograd import gradcheck

def my_op(x):                          # your shiny new operation
    return torch.sin(x) * x**2

x = torch.randn(4, dtype=torch.float64, requires_grad=True)
print(gradcheck(my_op, (x,)))         # True — backward() is correct
```

And what does the referee compare `backward()` against? **This**:

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h} \qquad \text{— a slope, measured with a tiny nudge}$$

Three puzzles hiding in that one call

The most sophisticated derivative engine on earth is graded by... a secant from first-year calculus. Look closer and it gets stranger:

1. **Trust** — the referee is an *approximation*. How wrong is it? Can it even be trusted to judge?
2. **The magic numbers** — `gradcheck` *refuses* float32, defaults to `eps = 1e-6`, and insists on the symmetric formula. Who chose those, and why?
3. **The scandal** — if nudging is good enough to referee, why doesn't PyTorch simply *use* it to train?

All three fall out of one lecture. The answers involve L2′s machine epsilon, L7′s Taylor remainder — and a number system where $\varepsilon^2 = 0$.

Today's one idea

On a computer you can approximate a derivative — and fight floating point — or compute it exactly, by making every number carry its own derivative.

Two acts:

- **Act I — the war**: nudge-and-divide fights truncation on one front, rounding on the other. We map the battlefield (a U-shaped curve) and find the least-bad step size ⭐.
- **Act II — the escape**: dual numbers make the error *algebraically zero*. Forward-mode autodiff is that trick, industrialized.

Learning outcomes

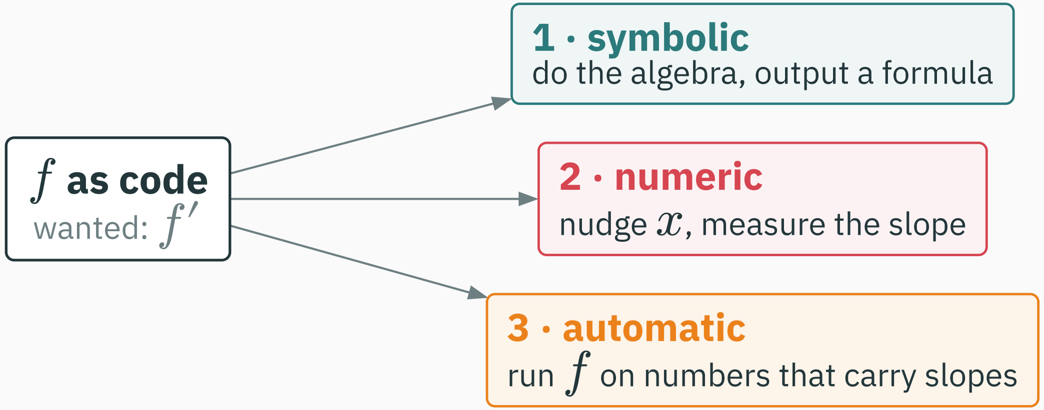
By the end of this lecture you will be able to:

1. Estimate derivatives with **forward and central differences** and predict their error orders.
2. Explain the **two-front war** — truncation vs rounding — and read an error-vs-h **U-curve**.
3. Derive the **optimal step** $h^* \approx \sqrt{\varepsilon}$ and the "half your digits" floor. ⭐
4. Explain why finite differences **cannot scale** to 10^6 parameters.
5. Compute with **dual numbers** $a + b\varepsilon$, $\varepsilon^2 = 0$ — and show they differentiate exactly.
6. Run **forward-mode autodiff** by hand: a (value, tangent) trace, one pass per input.

Three roads to a derivative

You have f. You want f'. Pick a road.

V VISUAL



This trichotomy — **symbolic / numeric / automatic** — organizes everything about differentiation on machines (it's the framing of the Baydin et al. survey in your reading).

Roads 1 and 2 you can guess. Road 3 sounds like magic. By the end of today it will feel obvious.

Road 1 · symbolic: do the algebra

What you did in L7, mechanized (SymPy, Mathematica): apply the rules, output a **formula**.

$$f(x) = x^2 \sin(x) \quad \to \quad f'(x) = 2x \sin(x) + x^2 \cos(x) \quad \text{— exact } \checkmark$$

Exact, human-readable… and it needs f to **be** a formula. Your training loss — Python loops, branches, lookups — has no closed form to hand over.

And formulas have a darker problem. Watch what happens when we **compose** — the one thing deep learning does all day.

Symbolic differentiation has a swelling problem

Iterate a humble map (composition = depth, exactly like stacked layers): $\ell_1 = x$, then $\ell_{k+1} = 4\ell_k(1 - \ell_k)$:

Depth	The formula
ℓ_2	$4x(1-x)$
ℓ_3	$16x(1-x)(1-2x)^2$
ℓ_4	$64x(1-x)(1-2x)^2\left(1-8x+8x^2\right)^2$

Now differentiate ℓ_4 — four compositions deep — and expand:

$$\mathrm{d}\ell_4/\,\mathrm{d}x = -131072x^7 + 458752x^6 - 638976x^5 + 450560x^4 - 168960x^3 + 32256x^2 - 2688x + 64$$

Expression swell: each level of composition multiplies the derivative's size. A 100-layer network's symbolic gradient would be astronomical.

Road 2 · numeric: just measure the slope

No algebra at all — **nudge and divide**:

```
def derivative(f, x, h=1e-8):
    return (f(x + h) - f(x)) / h     # works on ANY f — even a black box
```

- Three lines. No formula needed — loops, branches, whole programs welcome.
- The catch is one word: ≈. It's an **approximation** — and "how wrong?" turns out to be a genuinely great question.

That innocuous default `h=1e-8` is hiding a war story. By the ⭐ derivation you will know exactly where it comes from.

Road 3 · automatic: the one we're building toward

Automatic differentiation (AD): run the **program** of f, but on values that carry their own derivatives — every $+$ and $*$ updates both.

- **Exact** like symbolic — no h, no approximation, to the last bit
- **Formula-free** like numeric — works on code, never builds an expression
- Cost: a small constant times the cost of running f itself

Sounds too good to exist. It exists — `torch.autograd` *is* one. Today we build its **forward mode** by hand; L11 reverses it into backprop.

But first, road 2 deserves a fair trial. It loses beautifully.

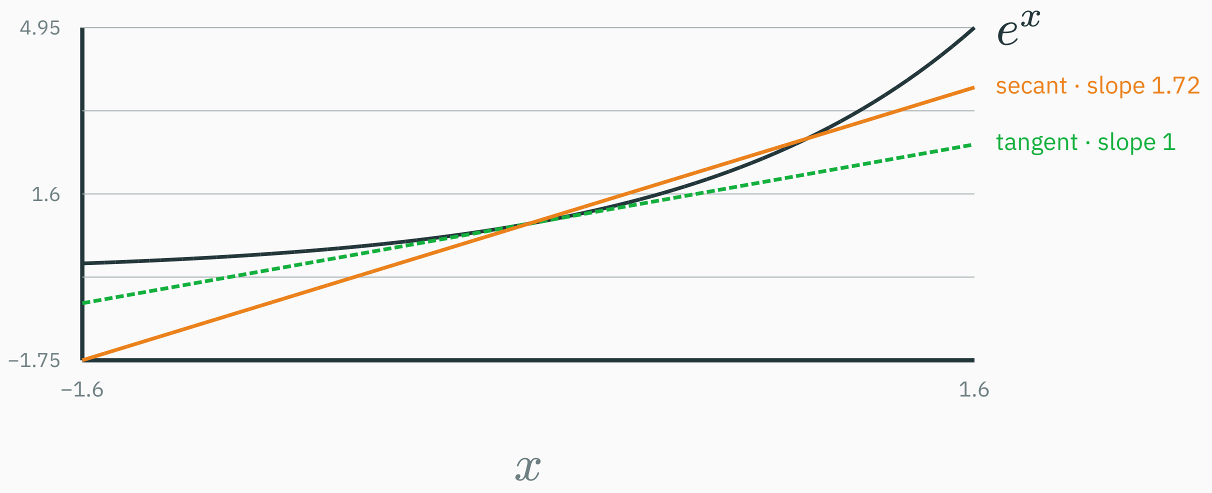
The forward difference: a war on two fronts

L7′s limit, with the limit amputated

V VISUAL

L7 *defined* the derivative as a limit of secant slopes. A computer can't take limits — it can only **stop early**:

$$f'(x) = \lim_{h\to 0} \frac{f(x+h) - f(x)}{h} \qquad \Rightarrow \qquad D_h f(x) := \frac{f(x+h) - f(x)}{h}$$



$f = e^x$ at $x = 0$, true slope 1: the secant with $h = 1$ overshoots to 1.72. Shrink h...

First contact: it kind of works

Test on $f = e^x$ at $x = 0$, where the truth is exactly $f'(0) = e^0 = 1$:

h	$D_h f(0)$ **(float64)**	**error**
10^{-1}	1.0517091808...	5.2 × 10^{-2}
10^{-2}	1.0050167084...	5.0 × 10^{-3}
10^{-3}	1.0005001667...	5.0 × 10^{-4}

Divide h by 10 → error divides by 10. Extrapolate the pattern: $h = 10^{-16}$ should give **16 correct digits**.

And look closer: the error isn't just *order* h — it is almost exactly $h/2$. That "2" is not a coincidence. Taylor knows why.

Front 1 · truncation: Taylor keeps receipts

L7 callback — Taylor's remainder form is an *equality* (some ξ between x and $x+h$): $f(x+h) = f(x) + hf'(x) + (h^2/2)f''(\xi)$. Rearrange the forward difference:

$$D_h f(x) = f'(x) + \underbrace{\frac{h}{2} f''(\xi)}_{\text{truncation error} \;—\; O(h)}$$

- For e^x at 0: $f'' \approx 1$ → error $\approx h/2$ — exactly the table's 5×10^{-3} at $h = 10^{-2}$ ✓
- Truncation $\propto h$: you stopped Taylor early and pay the curvature you ignored. Cure: **shrink h**.

So drive $h \to 10^{-16}$ and collect 16 digits… right? You spent L2 learning why not.

Front 2 · the subtraction shreds digits

L2 callback: catastrophic cancellation — subtracting near-equals promotes rounding garbage to the front. And $f(x+h) - f(x)$ is a subtraction of near-equals **by design**.

Take $h = 10^{-8}$. What float64 actually stores and computes:

```
e^h stored   =   1.0000000099999999392252903…  ← last digits: rounding noise
1.0          =   1.0000000000000000000000000
difference   =   0.0000000099999999392252903…  ← 8 leading digits annihilated
```

Both inputs carried ~16 good digits. Their difference carries **~8** — then `/ h` scales the surviving noise up by 10^8.

Started with 16 digits. Kept 8. The estimate: $0.9999999939\ldots$ — noise wearing a derivative costume.

Front 2 · the model: rounding error grows as ε/h

L2's rounding rule: each stored value is off by a relative δ, $|\delta| \le \varepsilon$ — recall $\varepsilon_{64} \approx 2.2 \times 10^{-16}$, $\varepsilon_{32} \approx 1.2 \times 10^{-7}$.

So each evaluation of f is really $f \cdot (1+\delta)$ — wrong by up to $\varepsilon\ |f|$. The numerator inherits up to $2\varepsilon\ |f|$ of garbage, and dividing by h **amplifies** it:

$$\text{rounding error of } \; D_h f \quad \approx \quad \frac{2\varepsilon\ |f(x)|}{h} \quad \text{— grows as } h \text{ shrinks!}$$

- Truncation shrinks with h; rounding **explodes** with $1/h$. Two enemies, opposite fronts.

Big h: truncation wins. Tiny h: rounding wins. Somewhere between: the least-bad h.

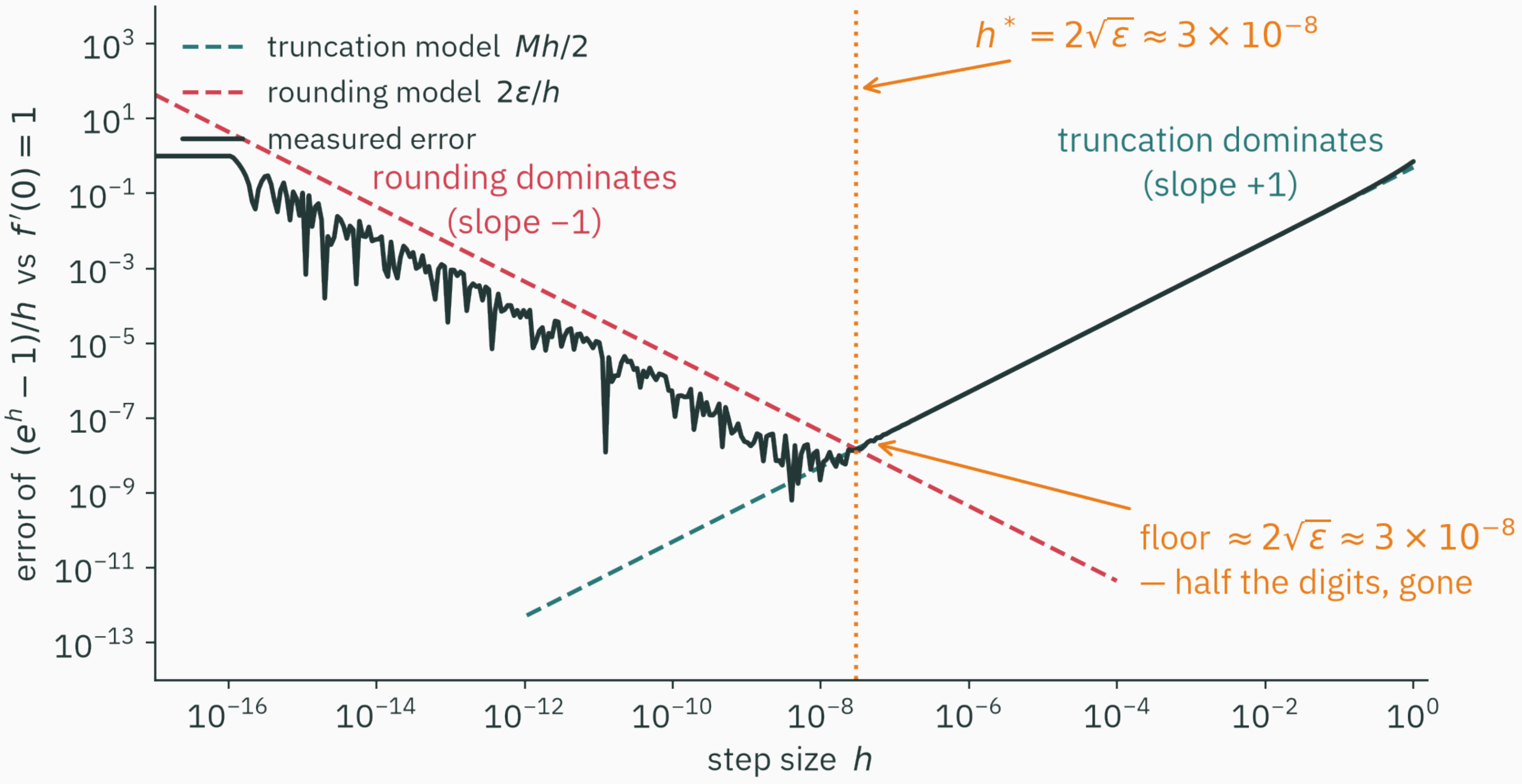
The war, live: every h loses somewhere

$f = e^x$, $x = 0$, float64 — every digit below is real:

h	$D_h f(0)$	**error**
10^{-4}	1.0000500017...	5 × 10^{-5}
10^{-8}	**0.9999999939...**	**6 × 10^{-9}** — the best it ever gets
10^{-12}	1.0000889006...	9 × 10^{-5} — worse again!
10^{-15}	1.1102230246...	1 × 10^{-1} — that's ε_{64}'s face
10^{-16}	**0.0000000000**	**100%**

The last row is pure L2: $1 + 10^{-16}$ **rounds to 1** — the nudge fell into the float gap, the numerator is exactly 0, and the estimator reports that the derivative of *everything* is zero. With total confidence.

V VISUAL



The whole war on one log–log plot: error vs h for $D_h e^x$ at $x = 0$, float64.

How to read a U-curve

- **Right wall, slope $+1$**: truncation, $\propto h$ — smooth, predictable, Taylor's receipt
- **Left wall, slope -1**: rounding, $\propto \varepsilon/h$ — jagged, because rounding noise is erratic, not smooth
- **The floor** near $h \approx 10^{-8}$: error $\approx 10^{-8}$ — the best this method will *ever* do in float64

There is no winning h — only a least-bad one. Every finite-difference scheme has a floor it cannot go below.

Where exactly is the floor, and the sweet spot under it? That deserves a real derivation — the lecture's ⭐.

⭐ The best possible step

⭐ Step 1: write the enemy down

D DERIVATION

Total error = truncation + rounding. Bound $|f''| \le M$ near x (curvature scale, L7), take $|f(x)| \approx 1$ for clean bookkeeping (our e^0 case exactly):

$$E(h) = \underbrace{\frac{M}{2}h}_{\text{truncation — grows with } h} + \underbrace{\frac{2\varepsilon}{h}}_{\text{rounding — grows with } 1/h}$$

- One term climbs, one falls: $E(h)$ is U-shaped — we **modeled** the measured curve
- Those are the two dashed lines drawn on the U-curve figure — this is a model you can check

Minimizing a function of one variable… we happen to own a lecture on that (L7).

⭐ Step 2: minimize — with L7's own calculus

D DERIVATION

Set the derivative of the error (the derivative of the error of a derivative!) to zero:

$$E'(h) = \frac{M}{2} - \frac{2\varepsilon}{h^2} = 0 \quad \Rightarrow \quad h^* = 2\sqrt{\frac{\varepsilon}{M}}$$

At the optimum, plug h^* back in — each enemy contributes **exactly half**:

$$E(h^*) = \sqrt{\varepsilon M} + \sqrt{\varepsilon M} = 2\sqrt{\varepsilon M}$$

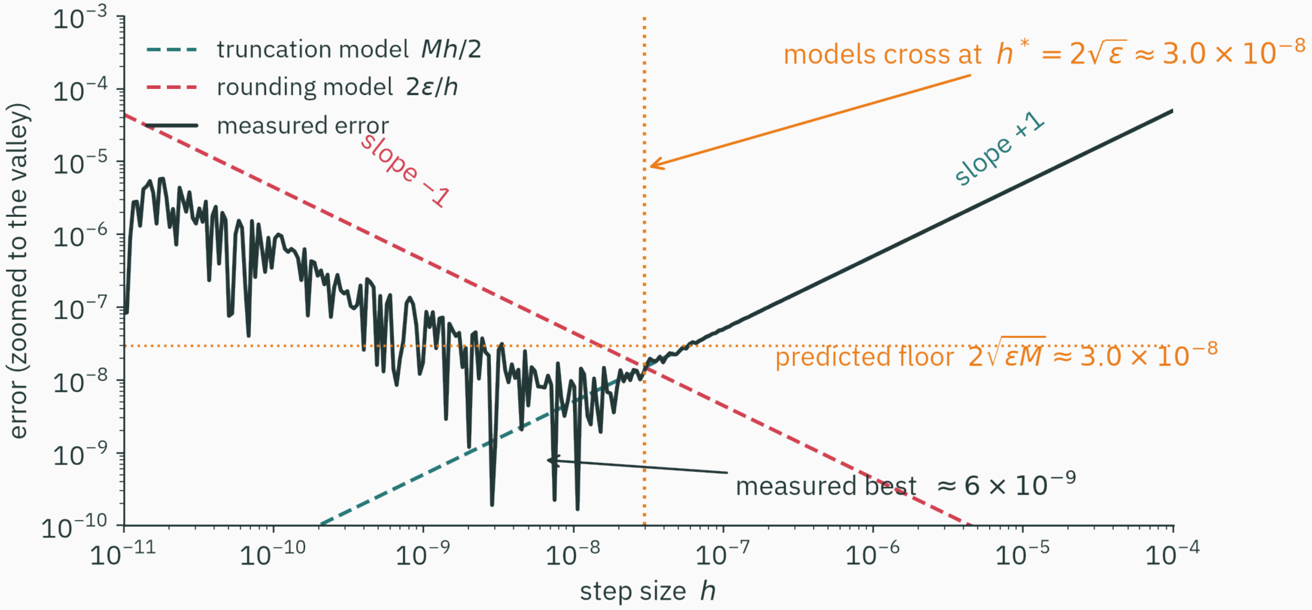
$h^* = 2\sqrt{\varepsilon/M} \approx \sqrt{\varepsilon}$, **best error** $\approx \sqrt{\varepsilon}$ **— for** $M \sim 1$**, both are "root machine epsilon".**

Float64: $h^* = 2\sqrt{2.2 \times 10^{-16}} \approx 3 \times 10^{-8}$, error floor $\approx 3 \times 10^{-8}$.

⭐ Step 3: verify against the machine

D DERIVATION   V VISUAL

Zoom into the U-curve's valley and lay the model on top of the measurement:



Walls cross at $h^* \approx 3 \times 10^{-8}$ ✓ — predicted floor 3×10^{-8} vs measured best 6×10^{-9} (worst-case bound, dead-on ballpark) ✓ — slopes ± 1 ✓. The two-term model *is* the whole story.

The floor is the headline: half your digits

Relative error $\sqrt{\varepsilon}$ means: **of the digits your float carries, forward differences keep half.**

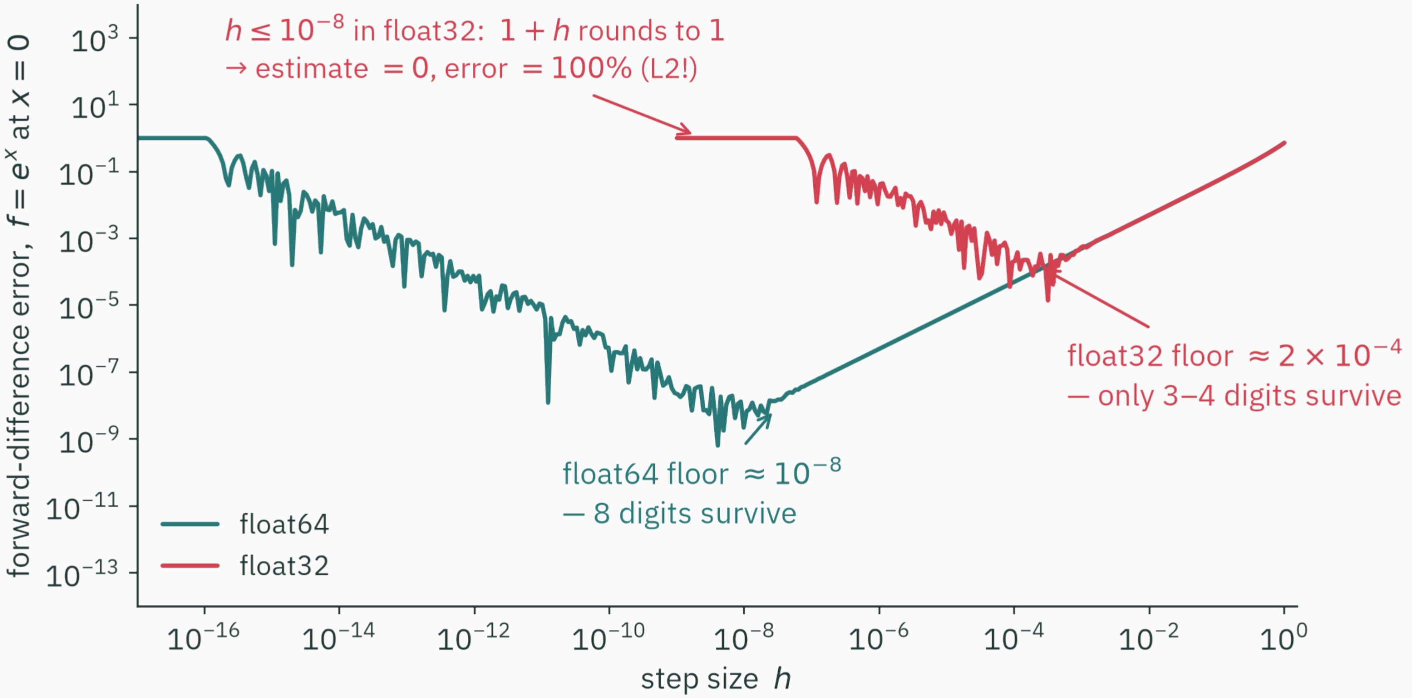
Format	**ε (L2's table)**	**digits carried**	**digits surviving $\sqrt{\varepsilon}$**
float64	2.2×10^{-16}	~16	~8
float32	1.2×10^{-7}	~7	~3.5
float16	9.8×10^{-4}	~3	~1.5 — noise

A float32 gradient check can't distinguish "correct" from "off by 0.1%". **That is why `gradcheck` refuses to run in float32** — puzzle 2, first half, solved.

Rule of thumb worth memorizing: finite differences cost you **half your precision**. Exactness isn't on the menu — at any h.

Same experiment, half the bits

V VISUAL



Same f, same code — float32's floor sits **ten thousand times higher**, and below $h = 10^{-8}$ its estimate is exactly 0: L2's `1.0 + 1e-8 == 1.0` demo, now with casualties.

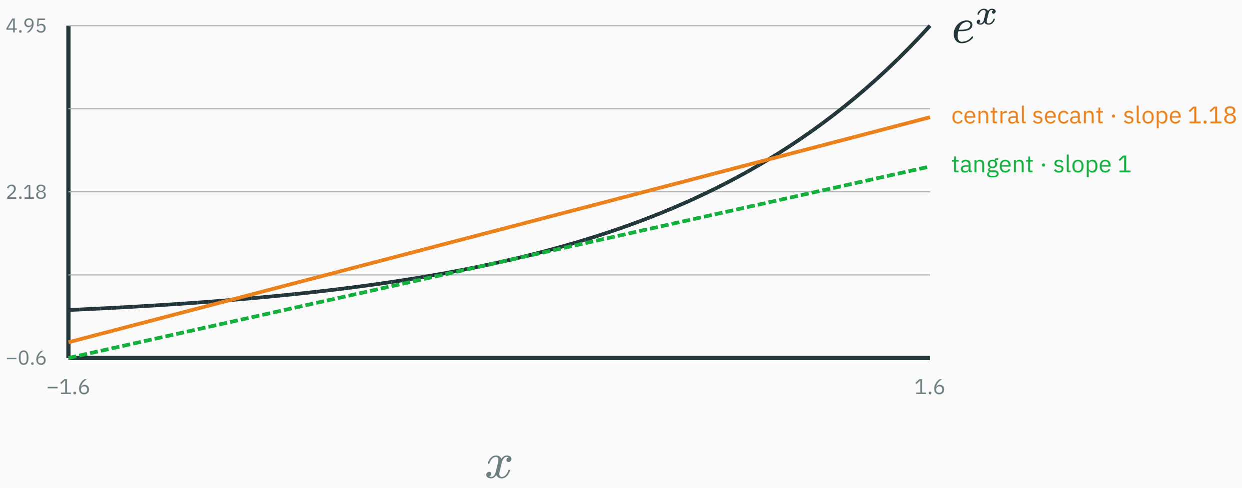
Central differences: a better secant

Symmetrize the secant

V VISUAL

Don't lean on one side — straddle the point: sample at $x - h$ **and** $x + h$.

$$C_h f(x) := \frac{f(x+h) - f(x-h)}{2h}$$



Same generous $h = 1$: the one-sided secant said 1.72; the symmetric one says 1.18 — already near-parallel to the tangent.

Why symmetry buys a whole order

Write Taylor **both ways** and subtract — watch the even terms die:

$$f(x+h) = f + hf' + \frac{h^2}{2}f'' + \frac{h^3}{6}f''' + \dots \qquad f(x-h) = f - hf' + \frac{h^2}{2}f'' - \frac{h^3}{6}f''' + \dots$$

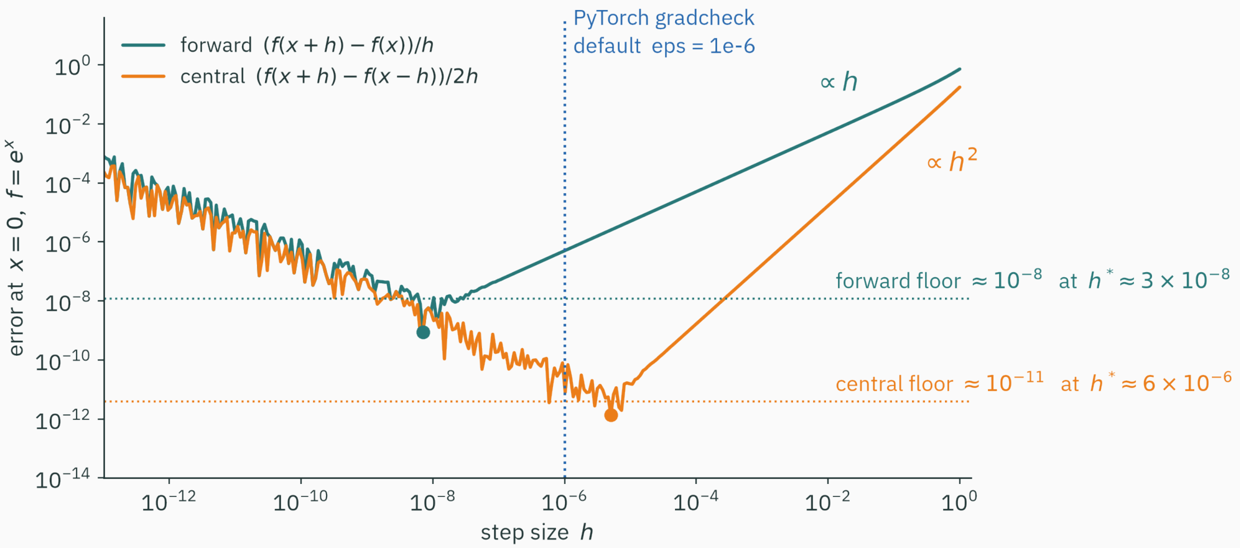
$$C_h f(x) = f'(x) + \underbrace{\frac{h^2}{6}f'''(\xi)}_{\text{truncation now } O(h^2)} \qquad \text{— the } f'' \text{ term cancelled } \textbf{exactly}$$

Halve $h \to$ **quarter** the truncation error. At $h = 10^{-2}$: forward is off by 5×10^{-3}, central by 1.7×10^{-5}.

Same two evaluations as forward (reused smartly), one order better.
This is why every serious gradient checker is central.

Better trenches, same war

V VISUAL



⭐ redo (practice): minimize $E(h) = Mh^2/6 + \varepsilon/h \rightarrow h^* \propto \varepsilon^{1/3} \approx 6 \times 10^{-6}$, floor $\propto \varepsilon^{2/3} \approx 10^{-11}$.

`eps = 1e-6` sits on central's sweet spot — every magic number was ε in disguise.

Checkpoint: the step-size war

Q QUESTION

Central differences, float64, well-behaved f. You shrink h from 10^{-5} to 10^{-7} and the error gets worse. What happened?

A. Truncation error grew — h is still too big

B. You crossed h^*: rounding's ε/h term now dominates

C. Central differences are only valid for $h > 10^{-5}$

D. The function must not be differentiable at x

Answer: the step-size war

A ANSWER

Correct: B — past h^*, rounding dominates

Why: Central's sweet spot in float64 is $h^* \approx 6 \times 10^{-6}$. Below it you slide **up the left wall** of the U-curve: truncation keeps shrinking ($\propto h^2$), but the cancellation term ε/h grows faster. Smaller h stopped being safer — that's the whole war in one move.

The wall: one evaluation per input

A gradient is n separate questions

For $f : \mathbb{R}^n \to \mathbb{R}$, the gradient wants $\partial f / \partial \theta_i$ for **every** i — and a nudge can only ask about **one input at a time**:

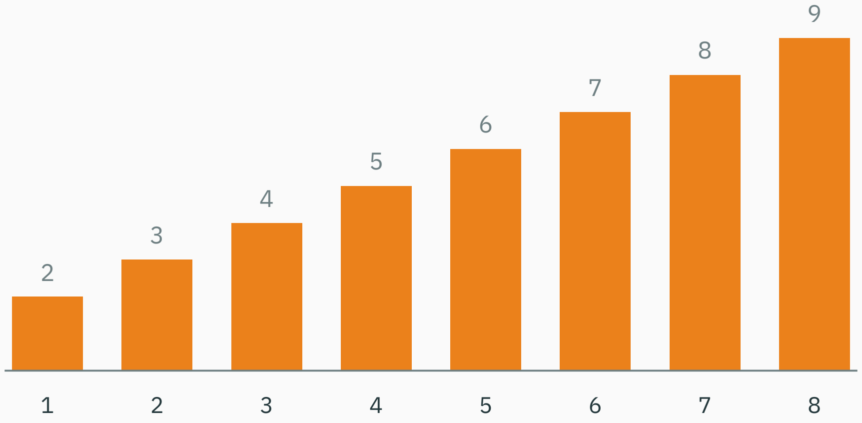
```
def grad_fd(f, theta, h=1e-6):
    g = np.zeros_like(theta)
    for i in range(len(theta)):          # one loop turn PER parameter
        e = np.zeros_like(theta); e[i] = h
        g[i] = (f(theta + e) - f(theta - e)) / (2 * h)
    return g                             # cost: 2n evaluations of f
```

Nudging θ_3 tells you **nothing** about θ_7. There is no bulk discount.

The bill grows with n

V VISUAL

forward-difference evaluations of f for one gradient, vs number of inputs n



The staircase is the point: $n+1$ evaluations (forward), $2n$ (central) — **linear in the number of parameters**, every single gradient step.

Now scale it to a real model

One forward pass of a big model ≈ 60 ms. One finite-difference gradient:

Model	**Parameters n**	**Time for ONE gradient**
linear regression	10	~0.7 s — fine
small MLP	10^5	1.7 hours
GPT-3	1.75×10^{11}	**~330 years**

Training needs **millions** of gradient steps. Finite differences don't need a better h — they need a different job.

Puzzle 3 solved: FD referees (one check, small tensors, run once) but can never play (every step, 10^{11} parameters).

Scoreboard after two roads

Route	**Accuracy**	**Cost for ∇f, $f : \mathbb{R}^n \to \mathbb{R}$**
symbolic	exact	expression swell; needs a formula
finite differences	$\approx$, floor $\sqrt{\varepsilon}$ — half your digits	$n+1$ evaluations
wanted	exact	~cost of running f, no formula

The “wanted” row looks greedy. The rest of the lecture collects it — starting from an idea that sounds like cheating: **invent a number**.

Dual numbers: an infinitesimal you can compute with

The wish: an h too small to square

Stare at why truncation existed at all:

$$f(x+h) = f(x) + hf'(x) + \underbrace{\frac{h^2}{2}f'' + \ldots}_{\text{everything we DON'T want}}$$

The unwanted terms all carry h^2 or higher. Finite differences make h^2 *small* and pay $\sqrt{\varepsilon}$ for it.

The wish: an h with h^2 **exactly zero** — yet $h \neq 0$ so slopes survive. No real number obeys that ($h^2 = 0 \Rightarrow h = 0$).

When reality lacks a number, mathematics invents one. $\sqrt{-1}$ scandalized algebra and gave us i. Today we decree a new one: ε, with $\varepsilon^2 = 0$.

Meet the dual numbers: decree $\varepsilon^2 = 0$

A **dual number** is $a + b\varepsilon$ — carry the pair (a, b), compute with ordinary algebra plus one new rule, $\varepsilon^2 = 0$:

Operation	**Result — just expand, then apply $\varepsilon^2 = 0$**
$(a + b\varepsilon) \pm (c + d\varepsilon)$	$(a \pm c) + (b \pm d)\varepsilon$
$(a + b\varepsilon)(c + d\varepsilon)$	$ac + (ad + bc)\varepsilon + \underbrace{bd\varepsilon^2}_{=0} = ac + (ad + bc)\varepsilon$

Look hard at the product's ε-part: $ad + bc$ — if b, d were "derivatives of a, c"... that is the **product rule**, appearing uninvited.

Two epsilons in this lecture, by historical accident: **machine epsilon** ε_{64} (a float's gap, L2) and this **dual unit** ε (an algebraic symbol). Unrelated beasts. From here on, ε means the dual unit.

First blood: $(3+\varepsilon)^2$

Feed $x = 3 + \varepsilon$ — “3, plus a formal nudge” — through plain squaring:

$$(3+\varepsilon)^2 = 9 + 6\varepsilon + \varepsilon^2 = 9 + 6\varepsilon$$

Read the pair: value $9 = 3^2$ ✓ and ε-coefficient $6 = 2 \cdot 3 = \left(\frac{\mathrm{d}}{\mathrm{d}x}\right)x^2 \,\big|_{x=3}$ ✓

Once more: $(3+\varepsilon)^3 = 27 + 27\varepsilon$ — and indeed $\left(x^3\right)' = 3x^2 = 27$ at $x = 3$. ✓

Push $x + \varepsilon$ through the arithmetic; the derivative falls out of the ε-slot.
No h. No limit. No subtraction of near-equals. Exact.

Why the ε-slot always holds the derivative

Not a coincidence — it's Taylor, with the tail **annihilated by algebra**. For smooth f, substitute a dual $t = b\varepsilon$ into the expansion:

$$f(a + b\varepsilon) = f(a) + f'(a)b\varepsilon + f''\frac{a}{2}b^2\underbrace{\varepsilon^2}_{0} + \ldots = f(a) + f'(a)\, b\, \varepsilon$$

Every term we fought as "truncation error" contains ε^2 — and ε^2 **is zero**. Nothing is truncated; nothing is left to truncate.

Finite differences make the Taylor tail small and pay $\sqrt{\varepsilon_{64}}$. Dual numbers make it zero — by decree.

Both fronts of the war just surrendered: no truncation (exact by algebra), no cancellation (no near-equal subtraction anywhere).

Every primitive learns one dual rule

$f(a + b\varepsilon) = f(a) + f'(a)b\varepsilon$ hands each primitive its own one-liner:

Primitive	Dual rule	...which is
$u + v$	$(a + c) + (b + d)\varepsilon$	sum rule
$u \cdot v$	$ac + (ad + bc)\varepsilon$	product rule
e^u	$e^a + e^a b\varepsilon$	chain rule for $\exp$
$\sin u$	$\sin a + (\cos a)b\varepsilon$	chain rule for $\sin$
$1/u$	$1/a - (b/a^2)\varepsilon$	reciprocal rule (practice)

A dozen rules cover a numerical library. In code this is **operator overloading**: teach $+$ and $*$ to a new class, and every program built from them differentiates itself.

A `Dual` class in ten lines

```
class Dual:
    def __init__(self, v, d):
        self.v, self.d = v, d                  # value, derivative
    def __add__(self, o):
        return Dual(self.v + o.v, self.d + o.d)
    def __mul__(self, o):
        return Dual(self.v * o.v,
                    self.v * o.d + self.d * o.v)   # product rule

x = Dual(3.0, 1.0)           # x = 3 + ε   (derivative of x wrt x is 1)
print((x * x).d)             # 6.0   — exact d/dx x² at 3
print((x * x * x).d)         # 27.0  — exact d/dx x³ at 3
```

Every $\star$ quietly applies the product rule. The “war” section of this lecture is simply *absent* from this code.

Forward mode: derivatives that ride along

The same recipe, now carrying two columns

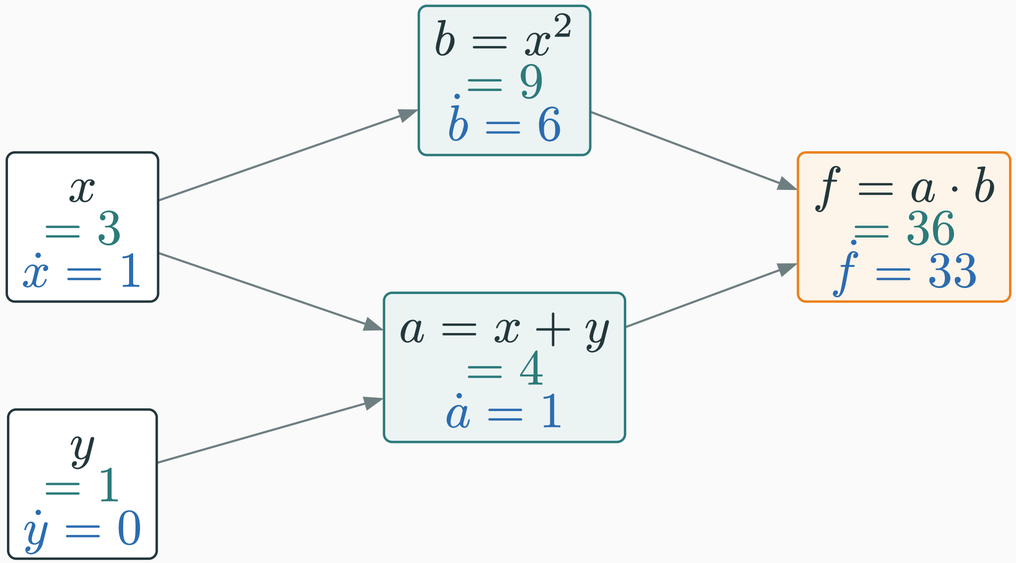
Forward-mode AD = evaluate f step by step, dual all the way. Take $f(x,y) = (x+y)\cdot x^2$ at $(3,1)$ — for $\partial f/\partial x$, **seed** $\dot{x}=1, \dot{y}=0$:

Step	**Value**	**Tangent rule**	**Tangent**
x (leaf)	3	seed	$\dot{x} = \mathbf{1}$
y (leaf)	1	seed	$\dot{y} = 0$
$a = x + y$	4	$\dot{a} = \dot{x} + \dot{y}$	1
$b = x^2$	9	$\dot{b} = 2x\dot{x}$	6
$f = a \cdot b$	36	$\dot{f} = \dot{a}b + a\dot{b}$	$9 + 24 = \mathbf{33}$

$f = 36,\ \partial f/\partial x = 33$ — **exact, one sweep, two columns.**

Tangents ride the same arrows

V VISUAL



- Values (teal) flow left → right; tangents (blue) flow **with** them, through the **same** graph, in the **same** pass
- Hold this picture: L11 will send a different quantity through these arrows — **backward**

Where 33 came from: two routes, added

x reaches f two ways — through a and through b — and the tangent of the product added one contribution per route:

$$\dot{f} = \underbrace{\dot{a}b}_{\text{via } a: 1\cdot 9 = 9} + \underbrace{a\dot{b}}_{\text{via } b: 4\cdot 6 = 24} = 33$$

Sanity check by L8′s rules, on the closed form $f = x^3 + x^2 y$:

$$\frac{\partial f}{\partial x} = 3x^2 + 2xy = 27 + 6 = 33 \quad \checkmark$$

File away "**sum over routes**" — it is the multivariate chain rule wearing graph clothes, and it is the exact spot where L11′s backward pass will plug in.

The second input needs a second pass

$\dot{f}$ answered **one** question: sensitivity to x. For $\partial f / \partial y$, re-run with the other one-hot seed $\dot{x} = 0, \dot{y} = 1$:

Step	**Value**	**Tangent (new seed)**
$a = x + y$	4	$\dot{a} = 0 + 1 = 1$
$b = x^2$	9	$\dot{b} = 2 \cdot 3 \cdot 0 = 0$
$f = a \cdot b$	36	$\dot{f} = 1 \cdot 9 + 4 \cdot 0 = \mathbf{9}$

$\nabla f(3, 1) = (33, 9)$ **— exact to the last bit, in two passes: one per input.**

Keep $(33, 9)$ warm — L11 re-derives this exact pair by a very different route.

The code agrees, digit for digit

The ten-line `Dual` class, on the two-variable f — same seeds, same numbers:

```
def f(x, y):
    return (x + y) * x * x      # ordinary code — no rewrite needed

print(f(Dual(3.0, 1.0), Dual(1.0, 0.0)).d)   # 33.0   ∂f/∂x  (seed x)
print(f(Dual(3.0, 0.0), Dual(1.0, 1.0)).d)   # 9.0    ∂f/∂y  (seed y)
```

No `h` anywhere in this program — so nothing to tune, no U-curve, no floor. The trichotomy's third road is **real**.

Compare the referee: central differences at the best possible h would return $32.99999999\ldots$-ish. `Dual` returns the float nearest to **exactly 33**.

Forward mode in the wild

Real frameworks ship exactly this trick (as "jvp" — more in a moment):

```
import jax
f = lambda x, y: (x + y) * x**2
val, dfdx = jax.jvp(f, (3.0, 1.0), (1.0, 0.0))   # value 36.0, tangent 33.0
```

- The seed needn't be one-hot: seeding (v_1, v_2) computes $v_1 \partial_x f + v_2 \partial_y f = \nabla f \cdot \boldsymbol{v}$ — a **directional derivative** (L8 callback!)
- For vector-valued f, that dot product becomes $J\boldsymbol{v}$ (L9's Jacobian): a **Jacobian–vector product**, JVP — computed without ever building J

The bill, again: one pass per **input**

Exactness: won. Now recount the cost for ML's shape of function:

Function shape	**Forward-mode passes**	**Verdict**
$f : \mathbb{R} \to \mathbb{R}^m$ (one knob, many outputs)	**1**	perfect
$f : \mathbb{R}^n \to \mathbb{R}$ (a **loss**: $n = 10^9$ knobs, one number)	n — one per seed	330 years again

One seed vector per pass, seeds live at the **inputs** — dual numbers made each pass exact, but the **count** didn't move.

Forward mode wins the accuracy war and **still** loses the scaling war — for the one function shape deep learning cares about.

Checkpoint: dual arithmetic

Q QUESTION

In dual arithmetic, $(2+\varepsilon)^3$ evaluates to...

A. 8

B. $8 + 3\varepsilon$

C. $8 + 12\varepsilon$

D. $8 + 12\varepsilon + 6\varepsilon^2$

Answer: dual arithmetic

A ANSWER

Correct: C — $8 + 12\varepsilon$

Why: $(2+\varepsilon)^2 = 4 + 4\varepsilon$; then $(4+4\varepsilon)(2+\varepsilon) = 8 + 4\varepsilon + 8\varepsilon = 8 + 12\varepsilon$. The ε-slot holds $12 = 3 \cdot 2^2$ — exactly $(x^3)'$ at $x = 2$. Option D forgot the decree: any ε^2 term is **already** zero, not “small”.

The cliffhanger

`gradcheck`, re-read with today's eyes

```
gradcheck(my_op, (x,))    # x MUST be float64; default eps=1e-6; central formula
```

Design choice	Which is exactly...
central, not forward	$O(h^2)$ truncation — one order better, same evaluations
float64 only	float32's $\sqrt{\varepsilon_{32}}$ floor ($\sim 10^{-3.5}$) can't clear the test's tolerances
eps = 1e-6	central's float64 sweet spot $h^* \sim \varepsilon_{64}^{1/3}$ — the U-curve's flat bottom
tests, not training	$2n$ evaluations: affordable once on a tiny op, absurd per step

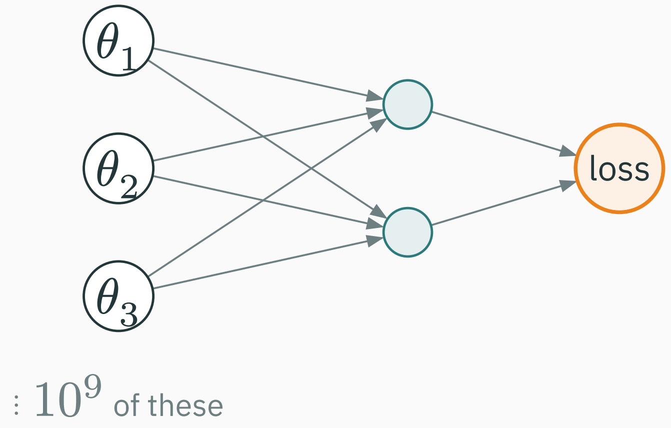
Gradient checking pairs the two roads: the approximate method referees the exact one. Puzzles 1–3, closed.

The scoreboard, complete

Route	**Verdict**	**Cost for ∇f, $f : \mathbb{R}^n \to \mathbb{R}$**
symbolic	exact, but expressions explode	—
finite differences	truncation vs rounding — both lose	n evals, **approximate**
forward-mode AD	**exact!** carries (value, derivative) pairs	n **passes** — one per input
reverse-mode AD	L11	?

Undefeated on accuracy, disqualified on cost. The last row is one lecture away — and its cost column is the reason deep learning exists.

Look at the shape of ML's function



- Forward mode seeds an **input** → the wide end: 10^9 seeds, 10^9 passes
- But the narrow end has exactly **one** node — the loss...

Teaser: seed the output once, and let derivatives flow backward through the same graph — one pass, all 10^9 partials. L11 builds it.

Interactive: three roads on one screen

I INTERACTIVE

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/autograd

The **autograd** article puts symbolic, finite-difference and autodiff side by side on a live computation graph. Rebuild today's $f = (x + y) \cdot x^2$, drag h and watch the U-curve war happen; then flip to the exact graph walk and see 33 and 9 appear — no h slider in sight.

Ten minutes before the next lecture: in the same article, press "reverse" on the graph walk and watch **which direction** the numbers travel. You'll have guessed most of L11.

Practice problems

Try on paper; verify in the T5 notebook (it pairs with L10–L11).

1. Estimate $\frac{\mathrm{d}}{\mathrm{d}x}\sin x$ at 0 with $h = 0.1$, forward and central. Both beat their advertised orders — which Taylor term vanished?
2. ⭐ redo for central: minimize $E(h) = Mh^2/6 + \varepsilon/h$; check h^* and floor against the figure.
3. Your GPU is float16 ($\varepsilon \approx 10^{-3}$). Can **any** h pass a 10^{-5}-tolerance gradient check? Use the $\sqrt{\varepsilon}$ rule.
4. Find $c + d\varepsilon$ with $(a + b\varepsilon)(c + d\varepsilon) = 1$; match your d to $(1/x)'$.
5. Trace forward mode on $g(x, y) = xy + x$ at $(2, 3)$, both seeds. Check: $\partial g/\partial x = y + 1$, $\partial g/\partial y = x$.
6. $n = 10^7$ parameters, 10 ms per pass: time one central-difference gradient, then one forward-mode gradient. (Both answers should make you want L11.)

⭐⭐⭐ Complex-step differentiation

★ OPTIONAL

A ghost of dual numbers hides inside ordinary complex arithmetic. Step **sideways**, off the real line:

$$f(x+ih) = f(x) + ihf'(x) - \frac{h^2}{2}f''(x) - i\frac{h^3}{6}f''' + \dots \quad \Rightarrow \quad f'(x) = \frac{\operatorname{Im}\, f(x+ih)}{h} + O(h^2)$$

The imaginary part starts at **zero** — extracting it subtracts nothing. No cancellation → no left wall → h can be absurdly small:

```
>>> np.imag(np.exp(0 + 1e-200j)) / 1e-200
1.0                                  # exact — with h = 10⁻²⁰⁰ (!)
```

$i^2 = -1$ only leaks into the **real** part at $O(h^2)$ — harmless. Dual numbers go one step further: $\varepsilon^2 = 0$ kills the leak entirely. (Squire & Trapp, 1998 — beloved in engineering codes that predate autodiff.)

Lecture 10 — summary

- **Three roads**: symbolic (swells) · numeric (approximate) · automatic (exact + formula-free).
- **The war**: truncation $Mh/2$ (L7) vs rounding $2\varepsilon/h$ (L2) — the U-curve and its floor.
- ⭐ $h^* \approx \sqrt{\varepsilon}$, best error $\approx \sqrt{\varepsilon}$ — **half your digits**; central improves to $\varepsilon^{2/3}$. Hence `gradcheck`: float64 · central · `eps = 1e-6`.
- **The wall**: one nudge per input → $n+1$ evals per gradient — ~330 years/step for GPT-3.
- **The escape**: $\varepsilon^2 = 0 \to f(a+b\varepsilon) = f(a) + f'(a)b\varepsilon$, exact by algebra; forward mode sweeps (value, tangent) once per **input** seed — $\nabla f(3,1) = (33,9)$ in two passes.

Read before L11 — MML §5.6 (start); Baydin et al., *AD in ML: a Survey*, §2–3 (skim — today's trichotomy is their map). **T5** drills the dual trace, then L11′s backward walk on the **same** graph.

Finite differences fight floating point — and both lose.

Next: **Reverse Mode & Backprop** — reverse the arrows: one pass for **all** inputs.